Data Management Project
DBMS Comparison

Leonardo Idone, 2088501

March 7, 2026

1 Introduction

In this document, we will qualitatively analyze the differences between mapping a dataset to the relational model and to a graph database model, specifically RDF. The dataset considered is IMDb's Non-Commercial dataset, composed of information regarding movies, actors, ratings, directors, writers and other attributes. As for technology chosen, we used Apache Jena Fuseki for the Triplestore and PostgreSQL for the Relational Database.

2 Methodology

2.1 IMDb's Non-Commercial Datasets

The dataset chosen has numerous problematic columns presenting a classic ETL problem that gets resolved during data ingestion in different ways:

- Many rows have null values, designated by the character `\N`, which needs to be handled cleanly.
- Columns like `primaryProfession` and `knownForTitles` have an array-like structure that does not map cleanly to either of our data models.
- Columns like `isAdult` and `isOriginalTitle` present boolean values in incompatible formats (0/1 instead of true/false).
- Some data is inconsistent; for instance, the `characters` column, where all strings are written in a single-element JSON array

Another problem is the volume of data, about 9 GiB of tsv files gzipped to 1.8 GiB. To effectively import them both smart buffering and efficient secondary memory usage need to be employed, together with maintaining an acceptable complexity during the ETL process, ideally $O(n)$.

2.2 Relational mapping

We will now focus on the reasoning behind the chosen tables and mapping to the respective columns.

For the file `title.basics` we have chosen to create the following relations:

```
titleType(titleTypeId, titleTypeName)
titleBasics(tconst, primaryTitle, originalTitle, isAdult, startYear, endYear, runtimeMinutes,
*titleTypeId)
genre(genreId, genreName)
titleBasics_genre(*tconst, *genreId)
```

The column `tconst` is a natural primary key for each title. Given the repetitive nature of `titleType`, the `tconst - titleType` one-to-many relationship sees the latter promoted to a new entity. The same happens for `tconst - genres`, with the difference that, being a many-to-many relationship it will need a junction table other than a dedicated `genre` relation. The rest of the columns act as attributes of the `titleBasics` relation.

For the file `name.basics` we have chosen this subsequent relations:

```
nameBasics(nconst, primaryName, birthYear, deathYear)
knownForTitles(*nconst, *tconst)
profession(professionId, professionName)
```

```
profession_nameBasics(*professionId, *nconst)
```

A string `nconst` is akin to `tconst` but for names, making it a natural primary key. The columns `primaryProfession` and `knownForTitles` are many-to-many relationships between a `name` and multiple `profession`(s) and `title`(s), so we follow the same logic as before: we create a relation to avoid duplicates and insert junction tables.

The data in `title.akas` has this relational schema:

```
titleAkas(akasId, *tconst, ordering, title, isOriginalTitle, *regionId, *languageId)
region(regionId, regionName)
language(languageId, languageName)
akaType(akaTypeId, akaTypeName)
akaType_titleAkas(*akaTypeId, *akasId)
attribute(attributeId, attributeText)
attribute_titleAkas(*attributeId, *akasId)
```

We denote *akas* as a way of calling alternative titles for a movie; for simplicity we decided to introduce an `akasId` to distinguish them. This is a many-to-one relationship to `titleBasics`. We also have the one-to-many relationships with `region` and `language`. And finally, two many-to-many relationships with `attributes` and with a new entity called `akaType`.

As for `title.principals` we implemented this schema:

```
titlePrincipals(principalsId, *tconst, ordering, *nconst, *categoryId, *jobId, *characterId)
character(characterId, characterName)
category(categoryId, categoryName)
job(jobId, jobName)
```

Title principals track what each person has done for a certain title. What a person (`name`) has done can change a lot between actor, director, camera operator and other roles. To track these roles we have two one-to-many relationships in `character` and `category`. If the job also can be correlated with a character (for example in the case of an actor) we use the one-to-many relationship `character`.

The table `title.crew` is very similar to `title.principals` but focuses on the roles of writers and directors:

```
writer(*nconst, *tconst)
director(*nconst, *tconst)
```

Since these are effectively two many-to-many relationships between `titles` and `names` all we have to do is create two junction tables between them.

The data in `title.episode` is instead about distinguishing which `titles` are singular movies and which are part of a series:

```
titleEpisode(episodeId, *tconst, *parentTconst, seasonNumber, episodeNumber)
```

As such we just need to create two one-to-many foreign keys on the relation: one for the `title` we are representing, and one for the `title` representing the whole series. We also track the season and episode number. Again we add an `episodeId` for simplicity.

Finally we map the `title.ratings`:

```
ratings(*tconst, averageRating, numVotes)
```

This maps perfectly to a separate relation `ratings` with a foreign key on the considered `title` and attributes on the number of votes and average rating.

To import the data to PostgreSQL we have chosen to use Python with the `psycopg` library. This allows direct piping of buffered rows to the RDBMS via COPY protocol, allowing fast transactional operations run while temporarily disabling integrity constraints until the transaction is finished. As another expedient to speed up the ingestion of data we have opted to create `unlogged` tables, which are not crash resistant but fine for the purpose of our project. We also used `rapidgzip` to overcome the single threaded, small buffering of data done by the builtin python `Gzip` library. The PostgreSQL library Psycopg uses a default buffer size of 64 KiB for its operations, which did not act as a bottleneck for the transactions. We implemented the strategy pattern on the various ways rows should be ingested by each different file and table, opening one connection for each separate table, one file at a time, committing the whole converted file at once. This approach, while fast enough, could be problematic for databases that refuse multiple concurrent connections from one host. A better way would be to generate the non-transactional data to be loaded sequentially in the database.

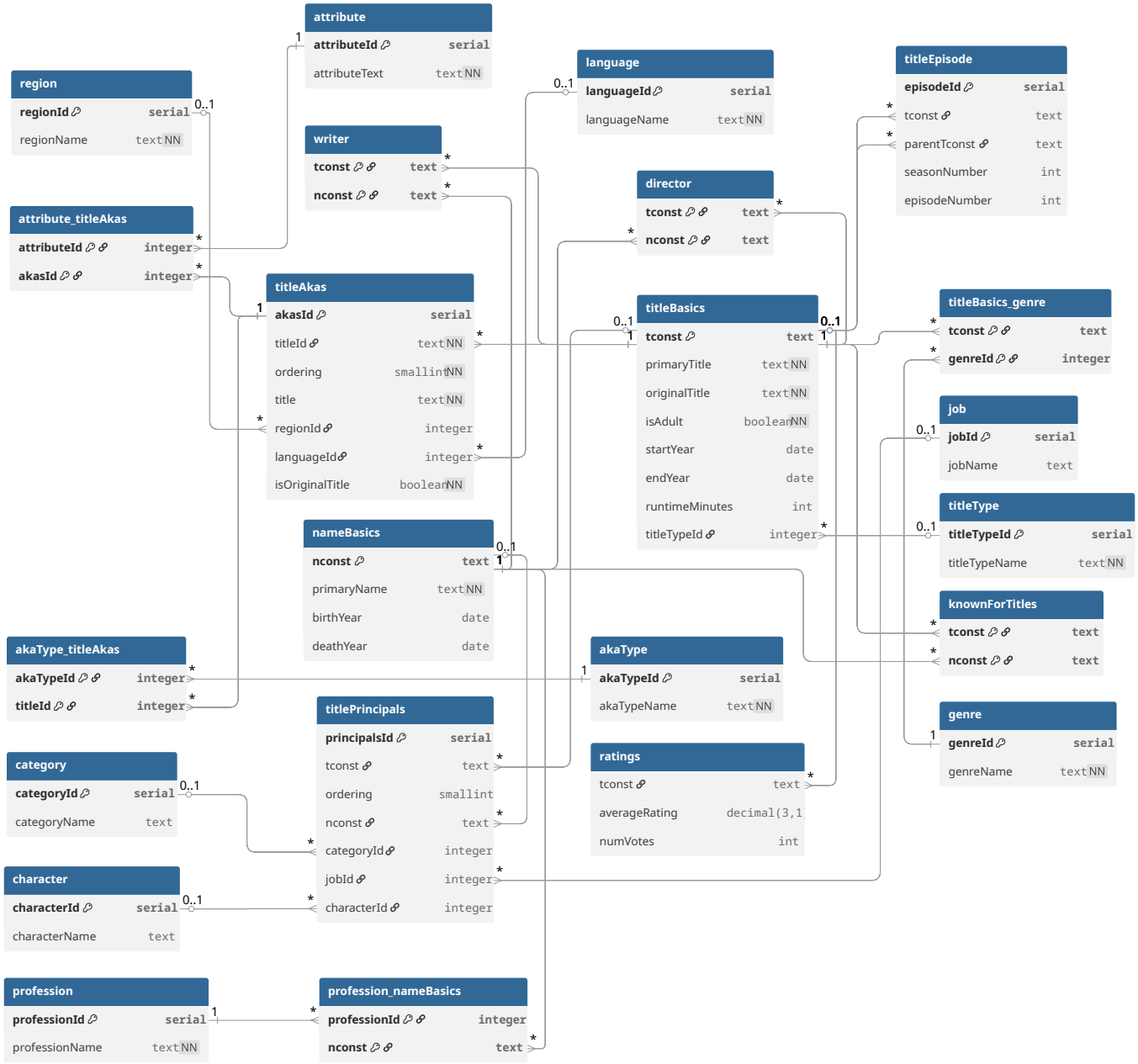

Figure 1: IMDb's Relational Schema (note: some column names have been renamed for implementation reasons).

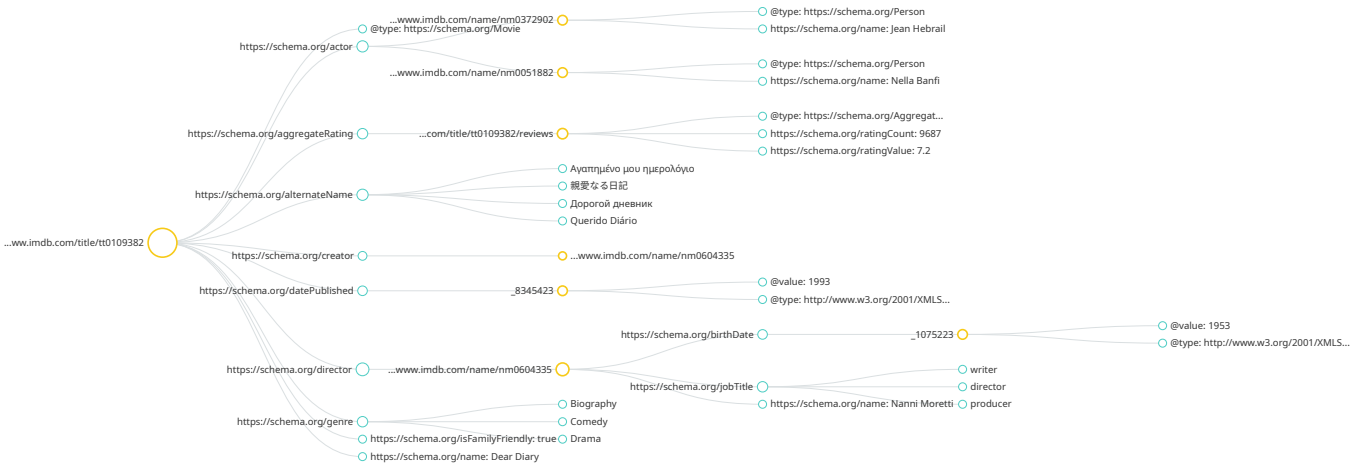

Figure 2: Excerpt of RDF Graph for tt0109382.

2.3 Triplestore Mapping

We will explain the rationale behind the approach used to build the Triplestore mapping. Since RDF is based on a way to allow interconnection of elements primarily in the web, and seeing how IMDb already implements it on its website, we decided to mimic its structure where possible. In particular, IMDb makes heavy use of `schema.org` models. On our part, in addition to mimicking IMDb's choices, where possible we decided to expand it with appropriate predicates and objects present in the most adaptable schema. Specifically, we used the `RDF:type` of `schema.org/Movie` to describe the `titles`, `schema.org/Person` for the `names`, `schema.org/AggregateRating` for the `ratings`. While IMDb's implementation of the schema lacked a node identifier for each triple, we included the subject by simply specifying the correct IMDb page for each node. For example, a `title` with a `tconst` will be mapped to the self-imposed IMDb schema (based on their website mapping) of `www.imdb.com/title/<tconst>`. Same thing for `schema.org/AggregateRating`, which will map to `www.imdb.com/title/<tconst>/reviews`. For every single other column seen in the dataset, these will map to predicates present in the aforementioned schemas used by IMDb itself. Some notable exceptions are language and region, which are not tracked in conformance with W3C's system of country codes, and as such are not specified when considering alternative titles (even though `schema.org` provides schemas for specifying detailed alternative names, just not with IMDb's country codes). IMDb doesn't bother tracking these country codes in their RDF schema, since they use it for the semantic web and not archival purposes. Apart from that lost information, everything was accounted for by `schema.org` types. One thing of note is that any null value we came across simply was not added to the graph.

To import the data we did not have any access to fast transactional interfaces to Apache Jena Fuseki, so to ingest it we had to extend the previous approach implemented in the relational part of this etl process in two parts:

1. Read all the files row by row, and convert them to a file format for fast parsing. In our case we chose N-Triples. One possibly better alternative is RDF Binary, also called RDF Thrift.

2. Import them in a non-transactional way to Apache Jena Fuseki.

The first part required us to build in Java the same strategy pattern implemented by our SQL solution, switching from transactionally copying to tables to non-transactionally dumping constructed N-Triples. To do so we leveraged Apache Jena's `StreamRDF` interface for fast writing together with Java's builtin `BufferedReader` and `GZIP*Stream`, with `FastCSV` for quick parsing and a buffer of 64 KiB where configurable (this didn't speed up the process much compared to the default 8 KiB buffer, the `ext4` filesystem in use has a block size of 4 KiB). For the row generation we used the `Node` and `NodeFactory` API for low-level construction of triples. All operations are single threaded since the performance is acceptable even without concurrent processing. It should be trivial to speed it up even more by processing multiple files at the same time.

The second part, given the number of 225 million tuples, required leveraging the large scale data loading tool `tdb2.xloader`. Created by Apache, it works orders of magnitude better than the standard tool `tdb2.tdbloader`. Their difference is the way they build their SPO, POS, OSP indexes for subjects, objects and predicates; `tdb2.xloader` does them sequentially instead of in parallel (SPO→POS→OSP), trying to maximize buffer usage, cache hits and using N-1 thread for parallel processing of a index at a time. This is a better approach for large amounts of data.

3 Analysis

The implementation of the two technologies was significantly different. Excluding the technological differences in loading the data, adapting the original data to the schemas was much more difficult in a relational setting than in a graph one. This is for one simple reason: relational schemas are very granular and detailed, always allowing us to find a way to map existing information to it. As such, for every problematic column we found there was an approach that allowed us to insert it efficiently in our schema.

Graph databases based on RDF instead are very standardized. While technically schema-less, international conventions on how to store and share data are present and encouraged in their use. These are very comprehensive and rarely lack any desired fields, speeding up the engineering and analysis phase immensely. Still, they require integrating a private, personal schema for unimplemented requirements, such as the custom country codes and language codes used by IMDb. This is faster and simpler than building everything from scratch.

Another important difference is the lack of native support to implement integrity checks at the schema level. Unlike SQL, RDF does not provide a native way to do this, and W3C has since fixed this flaw by introducing SHACL, a validation protocol for RDF graphs. This is completely decoupled from the triples themselves and as such is effectively separate from the data storage. This is expected, since RDF and graph databases were first and foremost designed for rapidly changing, extensible models and not archival needs. In other words, we are able to set constraints, but these are not directly correlated to a schema if not via external documentation.

Instead, SQL allows for very precise integrity checks at the schema level, declarable directly in the table definition. This enhances the integrity of stored data since we can leverage both an external validator and an internal checker.

Regarding the speed of queries, the simple model with which a graph database is built allows for the construction of very thorough triple-based indexes. These cover any association we may ever need regarding subject-predicate-object queries, which are the vast majority of query executed.

Conversely, SQL does not come with any indexes outside ones built for every primary key in every table we have. Secondary indexes need to be built separately. In theory, we could map the most common queries required by us and build what we need. But the construction of multiple indexes can be costly and must be considered carefully, making graph databases overall more versatile.

For example, comparing the same query for getting all the movies made by one director, selected by director name, their title and their year of publishing, both Apache Jena Fuseki and PostgreSQL spend similar time to get a result, as long as the relational database has an index on `nameBasics.primaryName`. As such, graph databases allow for faster development and expansion of functionalities, since implementing a new query for a service may not need any external intervention from the engineer other than writing the SPARQL code.

```
SELECT ?creativeWork ?directorName
       ?workName ?date
WHERE {
  VALUES ?directorName { "Nanni Moretti" }
  ?director schema:name ?directorName .
  ?creativeWork schema:director ?director
      ;
    schema:name ?workName ;
    schema:datePublished ?date .
} ORDER BY ?date
```

Listing 1: SPARQL Query

```
-- Index on nb.primaryname suggested
SELECT 'https://www.imdb.com/title/'
       || tb.tconst,
       nb.primaryname,
       tb.primarytitle,
       tb.startyear
FROM namebasics nb
JOIN director d
  ON nb.nconst = d.nconst
JOIN titlebasics tb
  ON d.tconst = tb.tconst
WHERE nb.primaryname = 'Nanni Moretti'
ORDER BY tb.startYear ASC
```

Listing 2: SQL Query

4 Conclusion

Overall, we can say that graph databases, in this case specifically RDF triplestores, provide a strong alternative to existing solutions in the market. While SQL is inherently capable of addressing these needs, it requires significantly more engineering. The aspects that are better in SQL are exactly this granular quality and built-in integrity constraints, which are directly implemented in the schema and effectively pair well with other client and server side validators connected to the database. It is hard to think of a project in which SQL really does not fit as a model. The worst that could happen is that SQL is not automatically as performant natively, it needs lots of engineering to be a good fit.